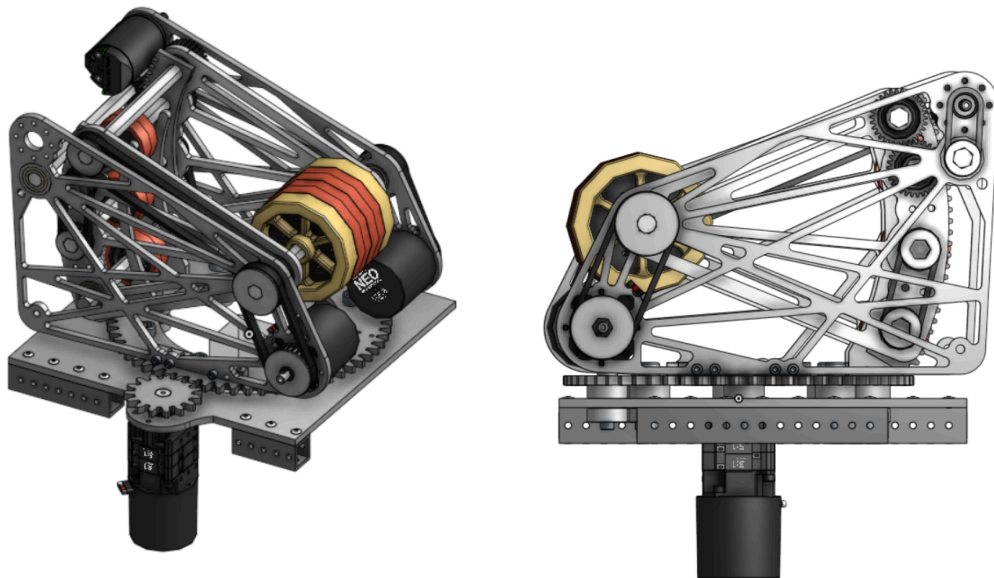


Situación Problema CM-S32601

Módulo Torreta (Turret)



Objetivo de la Práctica

Desarrollar el control de software para una torreta de un robot de FRC utilizando el patrón de diseño IO (Hardware Abstraction) del MARS Framework. El estudiante deberá programar la interfaz de datos, la capa de hardware físico (SparkMax), la capa de simulación y el módulo lógico principal, aislando el mecanismo por completo del resto del robot.

1. Especificaciones Mecánicas y Constantes (Proporcionado)

El equipo de mecánica ha instalado la torreta con las siguientes características. Se te entregará el archivo `TurretConstants.java` con todos los valores listos para usar:

- **Motor:** NEO Brushless controlado por un SparkMax (ID: 19).
- **Transmisión (Gear Ratio):** 20:1 (El motor gira 20 veces por cada vuelta completa de la torreta).
- **Límites Físicos:** La torreta no puede dar vueltas infinitas por el cableado. Tiene topes físicos a los -85° y $+85^\circ$.
- **Control:** El lazo cerrado (PID) correrá directamente en el hardware.

2. Entrega 1: La Interfaz y los Datos (**TurretIO.java**)

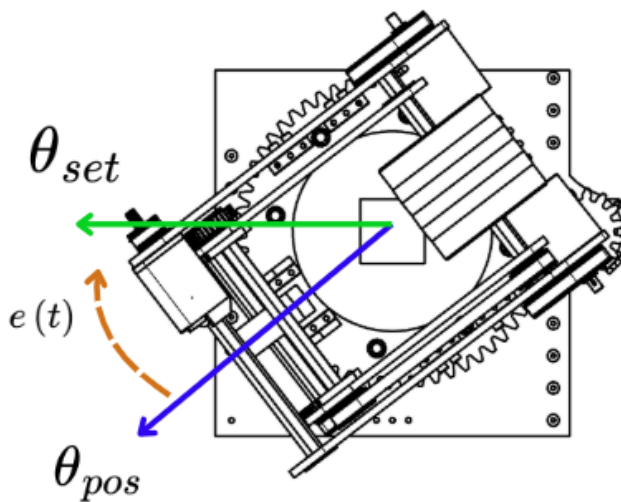
Tu primera tarea es definir el "contrato" de la torreta. Esto le dice al resto del robot qué datos podemos leer y qué comandos podemos enviar.

Requerimientos para la clase **TurretInputs (Hereda de **Data**):** Declara las siguientes variables públicas inicializadas.

- **angle** y **targetAngle** (Tipo **Rotation2d**).
- **velocityRPS** (Tipo **double**, en Rotaciones por Segundo).
- **appliedVolts** y **current** (Tipo **double**). *(Nota: Asegúrate de incluirlas en el método **snapshot()** para que el framework pueda clonarlas correctamente).*

Requerimientos para los métodos de control en **TurretIO:** Define las firmas (sin cuerpo) de los siguientes métodos:

- **setVoltage** (Recibe un **double**).
- **setPosition** (Recibe un **Rotation2d**).
- **setPositionWithFF** (Recibe un **Rotation2d** y un **double** para ArbFFVolts).
- **setSpeed** (Recibe un **double** para DutyCycle).
- **stop()** y **resetEnc()** (Sin parámetros).



3. Entrega 2: La Capa Física (Hardware Real y Simulación)

El framework requiere aislar el hardware. Deberás implementar la interfaz `TurretIO` en dos archivos distintos:

- **Archivo 1: `TurretIOSparkMax.java` (Hardware Real)**
 - Instancia el `SparkMax` y obtén su `RelativeEncoder`.
 - Configura los **Soft Limits** (Límites por software) usando `TurretConstants.kLowerLimit` y `kUpperLimit` para que el motor se proteja a sí mismo.
 - Configura los factores de conversión de posición y velocidad basándote en el Gear Ratio.
 - En `updateInputs()`, asegúrate de convertir las rotaciones del encoder a un objeto `Rotation2d` (invirtiendo el signo si la mecánica lo requiere) para llenar la clase `TurretInputs`.
- **Archivo 2: `TurretIOSim.java` (Simulación)**
 - Instancia un `SingleJointedArmSim` y un `ProfiledPIDController`.
 - En el método `updateInputs()`, calcula el voltaje si estás en modo posición (Closed Loop), limita el voltaje a $\pm 12V$ con `MathUtil.clamp`, y alimenta la simulación (`simMotor.update(0.02)`).
 - Llena los `TurretInputs` leyendo los radianes del simulador.

4. Entrega 3: El Módulo y la Telemetría (`Turret.java`)

Crea la clase `Turret` heredando de `ModularSubsystem<TurretInputs, TurretIO>`. Este es el núcleo lógico.

Retos de Implementación:

1. **Constructor Limpio:** Tu constructor solo debe recibir un parámetro: (`TurretIO io`). Pásalo a la clase padre (`super`) junto con el `KeyManager` y la telemetría. Define el comando por defecto como `Idle`.
2. **Registro de Telemetría:** Implementa la clase interna `TurretTelemetry` (que herede de `Telemetry<TurretInputs>`). Usa `KeyManager.TURRET_KEY` para publicar en `NetworkTables` datos vitales: Voltaje aplicado, Ángulo actual, Ángulo objetivo, Velocidad y Latencia.
3. **Método de Utilidad:** Crea un método `isAtTarget(double toleranceDegrees)` que compare el ángulo actual con el objetivo y retorne `true` si la diferencia es menor a la tolerancia.

5. Entrega 4: El Patrón Factory de Peticiones (**TurretRequest.java**)

En la arquitectura MARS, no usamos comandos sueltos, usamos "Requests" (Peticiones). Crea una interfaz decorada con **@RequestFactory** e implementa las siguientes clases internas (decoradas con **@CreateCommand**):

- **Idle**: El estado de reposo. Debe llamar a **actor.stop()** y retornar un **ActionStatus** indicando que la torreta está inactiva (**Turret.IDLE**).
- **manualControl**: Control directo por el piloto. Recibe un **DoubleSupplier** (el joystick).
 - *Regla de seguridad*: Si la torreta está dentro de los límites seguros (-90° a 90°), permite mover el motor enviando velocidad. Si se sale, manda velocidad 0 para detenerla. Retorna el status **Turret.MANUAL_CONTROL**.
- **Position**: Control de lazo cerrado. Recibe un **Rotation2d** objetivo y una tolerancia en grados.
 - *Lógica*: Envía la posición al hardware (**actor.setPosition**). Evalúa si la torreta ya llegó al objetivo. Si sí, retorna el status **Turret.LOCKED**; si no, retorna **Turret.TRACKING**.

6. Entrega 5: El Manifiesto y la Inyección (**Manifest.java**)

Para respetar la inyección de dependencias, el código nunca debe hacer **new TurretIOSparkMax()** directamente en la inicialización del robot.

Instrucción: En la clase **Manifest**, dentro del método **buildTurret()**, elimina cualquier referencia al chasis (drivetrain). Simplemente utiliza **Injector.createIO()** para seleccionar automáticamente el archivo de hardware correcto (Real, Simulación o Fallback) y retorna la nueva instancia: **return new Turret(io);**

7. Entrega 6: Integración con el Piloto (**OperatorBindings.java**)

Asigna los controles para que el operador humano pueda mover el mecanismo.

Instrucción: Dentro de **OperatorBindings**, configura un *Trigger* para que, cuando el operador mueva el joystick izquierdo en el eje X (**leftStickXTrigger**) y simultáneamente mantenga presionada la flecha derecha del D-Pad (**pov.right()**), se envíe la petición manual a la torreta:

```
superstructure.getTurret().setControl(() ->
TurretRequestFactory.manualControl().joystick(leftStick.x()))
```

8. Entrega 7: Automatización y Pruebas Unitarias (TurretTest.java)

Verifica que tu lógica de lazo cerrado funciona sin intervención humana creando un Test.

La Secuencia de Pruebas (TestRoutine): Crea una clase `TurretTest` anotada con `@MARSTest`. Programa un comando secuencial (`Commands.sequence`) que haga lo siguiente:

1. Envía una petición de `Position` a **45 grados**.
2. Usa `waitFor()` para pausar la secuencia hasta que la torreta reporte que llegó al objetivo (usando tu método `isAtTarget`).
3. Usa `assertLessThan()` para validar matemáticamente que el error absoluto en radianes sea menor a 2.0. Si falla, el test debe arrojar el mensaje: *"High turret error on target 1"*.
4. Repite exactamente el mismo proceso moviendo la torreta a **-45 grados**.
5. Regresa la torreta a **0 grados (kZero)**, valida la precisión, y finalmente envíala al estado `Idle`.